

IFN647 Week 5 Workshop Tasks

This week focuses on developing practical skills for text indexing for information retrieval in Python. You will learn how to efficiently process a collection of documents stored in a folder using the `glob` module, enabling scalable file handling and automation. Building on this, you will design and implement a Python function to construct an inverted index for a given set of documents, a fundamental data structure for search engines. Finally, you will gain an understanding of query processing in Python, including how to retrieve and rank relevant documents based on user queries, providing a foundation for more advanced search and text analytics applications.

Task 1 For the given XML documents (in [w5_data-1.zip](#)), design a python function `index_docs()` to index them (please remove stop words and index stems only). The returned index should be a dictionary `{term:{docID1:freq1, DocID2:freq2}, ...}`

The following are a sample of outcomes, where '6146' and '741299' are the document id, respectively.

```
Index = {'argentin': {'6146': 1}, 'bond': {'6146': 1},
'slight': {'6146': 2}, 'higher': {'6146': 1}, 'small':
{'6146': 1}, 'technic': {'6146': 2}, 'bounc': {'6146': 2},
'wednesday': {'6146': 1}, ..., 'time': {'741299': 1}, 'abl':
{'741299': 1}, 'push': {'741299': 1}, 'hard': {'741299':
1}, 'big': {'741299': 1}, 'third': {'741299': 1}, 'finish':
{'741299': 1}, 'porsch': {'741299': 1}, 'franc': {'741299':
1}, 'bob': {'741299': 1}, 'wollek': {'741299': 1},
'yannick': {'741299': 1}, 'dalma': {'741299': 1},
'belgian': {'741299': 2}, 'thierri': {'741299': 1},
'boutsen': {'741299': 1}, 'former': {'741299': 1},
'formula': {'741299': 1}, 'one': {'741299': 1}, 'driver':
{'741299': 1}, 'switch': {'741299': 1}, 'normal':
{'741299': 1}, 'share': {'741299': 1}, 'han': {'741299':
1}, 'stuck': {'741299': 1}, 'follow': {'741299': 1},
'power': {'741299': 1}, 'steer': {'741299': 1}, 'failur':
{'741299': 1}}
```

Task 2 Design a python function `doc_at_a_time(I, Q)`, where index I is a Dictionary of term:Dictionary of (itemId:freq), which returns a dictionary of docId:relevance for the given query Q (a term:freq dictionary).

Task 3 Design a python function `term_at_a_time(I, Q)`, where index I is a Dictionary of term:Dictionary of (itemId:freq), which returns a dictionary of docId:relevance for the given query Q (a term:freq dictionary).

Task 4 Design a python main program to call the above three functions for a query, e.g.,
 $Query = \{\text{'formula':1, 'one':1}\}$.

The sample outputs:

Document_at_a_time result-----
Document ID: 741299 and relevance weight: 2
...

Term_at_a_time result -----
Document ID: 741299 and relevance weight: 2
...